Assignment 2: λ-calculus and logic programming

This assignment has two parts, a theoretical part and a programming part. The theoretical part consists in proving properties about the λ-calculus extended with **product types** (also known as pairs or tuples). The programming part consists in writing a **Sudoku solver** in Prolog.

Theoretical part: proving properties of the λ-calculus with products

We consider the λ-calculus containing only arrow types, product types, and the unit type. The unit type is written $\mathbf{1}$ and it only has one element, written $\star$. The **grammar of types** is given by:

$$\tau, \sigma, \rho, \ldots ::= \mathbf{1} \mid \tau \to \sigma \mid \tau \times \sigma$$

The **grammar of terms** is given by:

$M, N, P, \ldots$	$::=$	x	variable
	$\mid$	$\lambda x : \tau. M$	lambda abstraction
	$\mid$	$M\, N$	application
	$\mid$	$\star$	unit constructor
	$\mid$	$\langle M, N \rangle$	pair constructor
	$\mid$	$M \Rightarrow \langle x, y \rangle. N$	pattern matching (pair eliminator)

The free occurrences of the variable x in M are bound by the lambda abstraction $\lambda x : \tau. M$. Similarly, the free occurrences of the variables x and y in N are bound by the pattern matching construct $M \Rightarrow \langle x, y \rangle. N$. We consider terms implicitly modulo α-equivalence, that is, renaming of all bound variables, and we write $\mathsf{fv}(M)$ of the set of variables that occur free in M.

The informal semantics of the pattern matching construct $M \Rightarrow \langle x, y \rangle. N$ is the following: first, evaluate M until it becomes a pair of values $\langle V, W \rangle$. Then continue evaluating the body N, binding x to V and y to W.

The **type system** is given by the following typing rules:

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash M : \tau}\texttt{T-VAR} \qquad \frac{\Gamma, x : \tau \vdash M : \sigma}{\Gamma \vdash \lambda x : \tau. M : \tau \to \sigma}\texttt{T-LAM} \qquad \frac{\Gamma \vdash M : \tau \to \sigma \quad \Gamma \vdash N : \tau}{\Gamma \vdash M\, N : \sigma}\texttt{T-APP}$$

$$\frac{}{\Gamma \vdash \star : \mathbf{1}}\texttt{T-UNIT} \qquad \frac{\Gamma \vdash M : \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash \langle M, N \rangle : \tau \times \sigma}\texttt{T-PAIR} \qquad \frac{\Gamma \vdash M : \tau \times \sigma \quad \Gamma, x : \tau, y : \sigma \vdash N : \rho}{\Gamma \vdash M \Rightarrow \langle x, y \rangle. N : \rho}\texttt{T-MATCH}$$

The grammar of **values** is given by:

$$V, W, \ldots ::= \lambda x : \tau. M \mid \star \mid \langle V, W \rangle$$

The **operational semantics** is given by the following evaluation rules:

$$\frac{M \longrightarrow M'}{M\, N \longrightarrow M'\, N}\texttt{E-APP1} \qquad \frac{N \longrightarrow N'}{V\, N \longrightarrow V\, N'}\texttt{E-APP2} \qquad \frac{}{(\lambda x : \tau. M)\, V \longrightarrow M\{x := V\}}\texttt{E-APPABS}$$

$$\frac{M \longrightarrow M'}{\langle M, N \rangle \longrightarrow \langle M', N \rangle}\texttt{E-PAIR1} \qquad \frac{N \longrightarrow N'}{\langle V, N \rangle \longrightarrow \langle V, N' \rangle}\texttt{E-PAIR2}$$

$$\frac{M \longrightarrow M'}{M \Rightarrow \langle x, y \rangle. N \longrightarrow M' \Rightarrow \langle x, y \rangle. N}\texttt{E-MATCH} \qquad \frac{}{\langle V, W \rangle \Rightarrow \langle x, y \rangle. N \longrightarrow N\{x := V\}\{y := W\}}\texttt{E-MATCHPAIR}$$

In the `E-MATCHPAIR` rule, we assume by α-renaming that $y \notin \mathsf{fv}(V)$.

Exercise 1. [Weakening]. Suppose that $\Gamma \vdash M : \tau$ holds. Let $z \notin \mathsf{dom}(\Gamma)$ be a variable not appearing in Γ and let ρ be any type. Prove that $\Gamma, z : \rho \vdash M : \tau$ must hold. Proceed by induction on the size of M.

Exercise 2. [Substitution]. Suppose that $\Gamma, x : \tau \vdash M : \sigma$ and $\Gamma \vdash N : \tau$ both hold. Show that $\Gamma \vdash M\{x := N\} : \sigma$ also holds. Proceed by induction on the size of M.

Exercise 3. [Type preservation]. Suppose that $\vdash M : \tau$ holds and that $M \longrightarrow M'$. Show that $\vdash M' : \tau$ holds. Proceed by induction on the size of M.

Exercise 4. [Progress]. Suppose that $\vdash M : \tau$ holds and that M is in normal form, that is, there does not exist a term M' such that $M \longrightarrow M'$. Show that M must be a value. Proceed by induction on the size of M.

Programming part: Sudoku solver in Prolog

A Sudoku puzzle is given by a 9×9 grid, partially filled in with numbers from 1 to 9. For example:

					1	2	3	
1	2	3			8		4	
8		4			7	6	5	
7	6	5						
						1	2	3
	1	2	3			8		4
	8		4			7	6	5
	7	6	5					

The goal of the puzzle is to fill each empty cell with a digit between 1 and 9, in such a way that:

- Each of the 9 rows contains no duplicate elements.
- Each of the 9 columns contains no duplicate elements.
- Each of the 9 distinguished 3×3 squares contains no duplicate elements.

We will represent a Sudoku puzzle in Prolog using a list of lists of elements. Each element may be a number between 1 and 9. A cell that starts empty will be represented by an uninstantiated variable. For example, the puzzle above may be reprented with the following list of lists:

```
[
  [X00, X01, X02, X03, X04,   1,   2,   3, X08],
  [  1,   2,   3, X13, X14,   8, X16,   4, X18],
  [  8, X21,   4, X23, X24,   7,   6,   5, X28],
  [  7,   6,   5, X33, X34, X35, X36, X37, X38],
  [X40, X41, X42, X43, X44, X45, X46, X47, X48],
  [X50, X51, X52, X53, X54, X55,   1,   2,   3],
  [X60,   1,   2,   3, X64, X65,   8, X67,   4],
  [X70,   8, X72,   4, X74, X75,   7,   6,   5],
  [X80,   7,   6,   5, X84, X85, X86, X87, X88]
]
```

Exercise 5. In the rest of this assignment:

- A **digit** is an integer between 1 and 9. A digit represents a cell that has been filled with a value.
- An **entry** is either a digit or an uninstantiated value. An entry represents either an empty cell or a filled cell.

Define the following predicates:

a) `isDigit(?X)` which holds if `X` is a digit between 1 and 9.

b) `isElement(?X)` which holds if `X` is an uninstantiated variable or a digit between 1 and 9. If `X` is an uninstantiated variable, this predicate should **not** instantiate it. You can use the predicates `var(X)` or `nonvar(X)` to check if `X` is an uninstantiated variable or not.

c) `emptySudoku(-Sudoku)` which creates a list of lists whose entries are all freshly uninstantiated variables, representing a 9×9 grid.

d) `isSudokuPuzzle(+Sudoku)` which holds if `Sudoku` is a valid Sudoku puzzle. A valid Sudoku puzzle is a list of lists of **entries**.

Exercise 6. Define the following predicates:

a) `entry(+Sudoku, +I, +J, -Entry)`: assuming as a precondition that `Sudoku` is a valid Sudoku puzzle and that $0 \leq \texttt{I} < 9$ and $0 \leq \texttt{J} < 9$, this predicate holds if `Entry` is the entry at row `I` and column `J` of the given Sudoku puzzle. For example, if `Sudoku` is the puzzle shown above, then:

```
?- entry(Sudoku, 2, 7, Entry).
Entry = 5.
?- entry(Sudoku, 3, 7, Entry).
Entry = X37.
```

b) `row(+Sudoku, +I, -Row)` assuming as a precondition that `Sudoku` is a valid Sudoku puzzle and that $0 \leq \texttt{I} < 9$, it returns the `I`-th row. For example, if `Sudoku` is the puzzle shown above, then:

```
?- row(Sudoku, 2, Row).
Row = [8, X21, 4, X23, X24, 7, 6, 5, X28].
```

c) `column(+Sudoku, +J, -Row)` assuming as a precondition that `Sudoku` is a valid Sudoku puzzle and that $0 \leq \texttt{J} < 9$, it returns the `J`-th column. For example, if `Sudoku` is the puzzle shown above, then:

```
?- column(Sudoku, 7, Column).
Column = [3, 4, 5, X37, X47, 2, X67, 6, X87].
```

d) `square(+Sudoku, +K, -Square)` assuming as a precondition that `Sudoku` is a valid Sudoku puzzle and that $0 \leq \texttt{K} < 9$, it returns the `K`-th distinguished 3×3 square. For example, if `Sudoku` is the puzzle shown above, then:

```
?- square(Sudoku, 7, Square).
Square = [1, 2, 3, 8, X67, 4, 7, 6, 5, X86, X87, X88].
```

Exercise 7. Define the following predicates:

a) `withoutDuplicates(+List)`: this predicate receives a list of entries and holds if and only if all the entries which are digits are different. For example:

```
?- withoutDuplicates([9, 7, X, 6, X, 1]).
true.
?- withoutDuplicates([9, 6, X, 6, X, 1]).
false.
```

Note that the predicate must ignore uninstantiated variables. You can use the predicates `var(X)` or `nonvar(X)` to check if `X` is an uninstantiated variable or not.

b) `solve(+Sudoku)`: this predicate receives a Sudoku puzzle (whose entries may be uninstantiated variables). The predicate should succeed if the puzzle has a solution, and fail if the puzzle has no solution. More precisely, it should generate all possible solutions to the puzzle, instantiating the entries into digits.

For example, if `Sudoku` is the puzzle shown above, then:

```
?- solve(Sudoku), show(Sudoku).
```

You can use the "`show`" predicate provided as part of the assignment. For example:

```
?- sudoku1(S), show(S).
+---+---+---+
|   |  1|23 |
|123|  8| 4 |
|8 4|  7|65 |
+---+---+---+
|765|   |   |
|   |   |   |
|   |   |123|
+---+---+---+
| 8 |4  |765|
| 76|5  |   |
|   |   |   |
+---+---+---+
S = ...


?- sudoku1(S), solve(S), show(S).
+---+---+---+
|657|941|238|
|123|658|947|
|894|237|651|
+---+---+---+
|765|123|489|
|231|894|576|
|948|765|123|
+---+---+---+
|512|376|894|
|389|412|765|
|476|589|312|
+---+---+---+
S = ...
```

Submission

- You must submit your solution using the moodle website.
- Your solution must be submitted as two separate files: a PDF file for the theoretical part and a single Prolog file for the programming part. The PDF file for the theoretical part can be typeset digitally with a word processor (for example, LaTeX) or it can be handwritten.
- The deadline for submission is **Thursday 2026-06-11 until 23:59:59**, Shantou time (GMT+8).
- **Late submissions** will be accepted until Friday 2026-06-12 23:59:59 with **10% penalty**.
- No submissions will be accepted after Friday 2026-06-12 23:59:59.
- Sometimes the course moodle experiences delays. Please **do not wait to submit until the last minute**.